CASE STUDY

Swiggy

SQL Analytics on Food Delivery Data

A relational-database project analyzing customers, restaurants, orders, deliveries, payments, and feedback using structured SQL queries.

OBJECTIVE:

To analyze Swiggy's food delivery data using SQL — covering customers, restaurants, orders, and delivery partners — through advanced joins and queries, in order to uncover insights on customer behavior, restaurant performance, and delivery efficiency to support strategic business decisions.

Made By: **Samyak Jain**

1

QUESTION 1

Display all customers who live in 'Delhi'.

query.sql

```
SELECT *
FROM customers
WHERE city = 'Delhi';
```

Insight: Filters the customer base down to a single city.

2

QUESTION 2

Find the average rating of all restaurants in 'Mumbai'.

query.sql

```sql
SELECT restaurant_id, name, city, AVG(rating)
FROM restaurants
WHERE city = 'Mumbai'
GROUP BY 1, 2, 3;
```

Insight: Aggregates restaurant ratings for one city.

3

QUESTION 3

List all customers who have placed at least one order.

query.sql

```
SELECT O.customer_id, C.name
FROM orders AS O
INNER JOIN customers AS C
  ON C.customer_id = O.customer_id
GROUP BY O.customer_id, C.name;
```

Insight: Joins customers to orders to isolate active buyers.

4

QUESTION 4

Display the total number of orders placed by each customer.

query.sql

```
SELECT O.customer_id, C.name,
       COUNT(order_id) AS Count_Of_Orders
FROM orders AS O
INNER JOIN customers AS C
  ON C.customer_id = O.customer_id
GROUP BY O.customer_id, C.name;
```

Insight: Counts orders per customer to gauge engagement.

5

QUESTION 5

Find the total revenue generated by each restaurant.

query.sql

```
SELECT R.restaurant_id, R.name,
       SUM(O.total_amount) AS Revenue
FROM restaurants AS R
INNER JOIN orders AS O
  ON R.restaurant_id = O.restaurant_id
GROUP BY R.restaurant_id, R.name
ORDER BY Revenue DESC;
```

Insight: Sums order value per restaurant, ranked highest first.

6

QUESTION 6

Find the top 5 restaurants with the highest average rating.

query.sql

```
SELECT restaurant_id, name, city,
       AVG(rating) Avg_Rating
FROM restaurants
GROUP BY 1, 2, 3
ORDER BY Avg_Rating DESC
LIMIT 5;
```

Insight: Ranks restaurants by average rating and keeps the top 5.

7

QUESTION 7

Display all customers who have never placed an order.

query.sql

```
SELECT C.customer_id, C.name
FROM orders AS O
RIGHT JOIN customers AS C
  ON C.customer_id = O.customer_id
WHERE O.order_id IS NULL;
```

Insight: A RIGHT JOIN with a NULL check surfaces inactive customers.

8

QUESTION 8

Find the number of orders placed by each customer in 'Mumbai'.

query.sql

```
SELECT C.customer_id, C.name, C.city,
       COUNT(*) AS Total_Order_Placed
FROM orders AS O
INNER JOIN customers AS C
  ON C.customer_id = O.customer_id
WHERE C.city = 'Mumbai'
GROUP BY C.customer_id, C.name, C.city;
```

Insight: Combines a city filter with a per-customer order count.

9

QUESTION 9

Display all orders placed in the last 30 days.

query.sql

```
SELECT *
FROM orders
WHERE order_date >= DATE_ADD(
  (SELECT MAX(order_date) FROM orders),
  INTERVAL -30 DAY
);
```

Insight: Uses the latest order date as an anchor for a rolling 30-day window.

10

QUESTION 10

List all delivery partners who have completed more than 1 delivery.

query.sql

```
SELECT DV.partner_id, DV.name,
       COUNT(*) AS Delivery_Count
FROM orderdelivery AS OD
INNER JOIN deliverypartners AS DV
  ON DV.partner_id = OD.partner_id
INNER JOIN deliveryupdates AS DU
  ON DU.order_id = OD.order_id
WHERE DU.status = 'Delivered'
GROUP BY DV.partner_id, DV.name
HAVING COUNT(*) > 1
ORDER BY Delivery_Count DESC;
```

Insight: Chains three tables, then filters delivered counts above 1.

11

QUESTION 11

Find customers who have placed orders on exactly three different days.

query.sql

```
SELECT C.customer_id, C.name,
       COUNT(DISTINCT O.order_date) AS Distinct_Order_Days
FROM orders AS O
INNER JOIN customers AS C
  ON C.customer_id = O.customer_id
GROUP BY C.customer_id, C.name
HAVING COUNT(DISTINCT O.order_date) = 3;
```

Insight: COUNT(DISTINCT ...) with HAVING pinpoints exactly 3 order days.

12

QUESTION 12

Find the delivery partner who has worked with the most different customers.

query.sql

```
SELECT DV.partner_id, DV.name,
       COUNT(DISTINCT O.customer_id) AS Unique_Customers
FROM orderdelivery AS OD
INNER JOIN deliverypartners AS DV
  ON DV.partner_id = OD.partner_id
INNER JOIN orders AS O
  ON O.order_id = OD.order_id
GROUP BY DV.partner_id, DV.name
ORDER BY Unique_Customers DESC;
```

Insight: Counts distinct customers per partner, ranked highest first.

13

QUESTION 13

Identify customers who share a city and ordered from the same restaurant, but on different dates.

query.sql

```
WITH OrderDetails AS (
  SELECT C.customer_id AS ID, C.name AS CustomerName,
         C.city AS City, R.name AS RestaurantName,
         R.restaurant_id, O.order_date
  FROM orders AS O
  JOIN customers AS C ON C.customer_id = O.customer_id
  JOIN restaurants AS R ON R.restaurant_id = O.restaurant_id
)
SELECT O1.ID, O1.CustomerName, O1.City, O1.RestaurantName
FROM OrderDetails AS O1
JOIN OrderDetails AS O2
  ON O1.City = O2.City
  AND O1.RestaurantName = O2.RestaurantName
  AND O1.order_date < O2.order_date
  AND O1.ID <> O2.ID;
```

Insight: A CTE self-joined on city, restaurant, and date finds matching pairs.

Thank You

13 SQL queries covering joins, aggregation, subqueries, and CTEs to analyze Swiggy's customers, restaurants, orders, and deliveries.

Prepared for: [Institute / Course Name] Prepared by: [Your Name]